

VPA IB DP CS — Topic 5: Abstract Data Structures (HL Only, Weeks 1 – 9)

Topic 5: Abstract Data Structures 抽象数据结构 Chōuxiàng Shùjù Jiégòu HL ONLY

IB Syllabus: Topic 5 (5.1 – 5.3) | Level: HL ONLY | Duration: 9 weeks (30 hours)

Topic 5.1: Stacks, Queues, Linked Lists, Trees, Graphs 栈、队列、链表、树、图

Week 1 — Linked Lists 链表 Liànbǎo

Lesson	Content	Activities	Differentiation
1	Linked list structure: nodes, data field, pointer/reference field, head, tail, null (节点, 指针, 头节点)	Draw linked list diagrams; trace traversal	Bronze: Label diagram; Silver: Trace operations; Gold: Compare to arrays
2	Singly linked list operations: insert at start, insert at end, delete, traverse, search	Write pseudocode for each operation; trace on diagrams	Bronze: Trace given code; Silver: Write pseudocode; Gold: Handle edge cases (empty list, single node)
3	Doubly linked lists: forward and backward traversal; advantages over singly linked	Modify singly linked pseudocode for doubly linked	All: Understand concept; Gold: Implement with error checking
4	Circular linked lists; sentinel nodes; comparison of all variants	Comparison table: memory, access time, use cases	All: Complete table; Gold: Analyse cache performance
5	TOK: A linked list is an abstraction with no physical existence. In what sense can we say it "exists"? Is it a mathematical object, a computational one, or both?	Discussion; written reflection	All: Contribute; Gold: Reference Platonism vs formalism

TOK Knowledge Question: A linked list is defined entirely by relationships (pointers) between nodes, not by the nodes themselves. Does this mean the structure has an existence independent of its contents? Can the form of a data structure be said to exist in a different way from its matter?

Week 2 — Stacks and Queues (Dynamic Implementation) 栈与队列 Zhàn yǔ Duìliè

Lesson	Content	Activities
1	Stack as linked list: push (insert at head), pop (remove from head), dynamic sizing (栈的动态实现)	Implement dynamic stack; compare to array-based

Lesson	Content	Activities
2	Queue as linked list: enqueue (insert at tail), dequeue (remove from head)	Implement dynamic queue; trace pointer updates
3	Circular queue implementation using arrays; full/empty detection	Design circular buffer for streaming data
4	Priority queue: concept, insertion by priority, dequeuing highest priority	Implement simple priority queue; discuss heap optimisation
5	Formative: Implement stack and queue operations in pseudocode (30 marks)	Timed; peer mark with criteria

Week 3 — Trees 树 Shù

Lesson	Content	Activities
1	Tree terminology: root, node, edge, parent, child, sibling, leaf, depth, height, subtree (根节点, 叶子节点, 深度, 高度)	Label tree diagrams; identify relationships
2	Binary trees: each node has at most two children; binary search tree (BST) property	Build BST from sequence; trace search operations
3	BST operations: insert, search, delete (leaf, one child, two children cases)	Write pseudocode; trace complex deletions
4	Tree balancing: why it matters; AVL and Red-Black trees (conceptual)	Analyse unbalanced vs balanced tree search times
5	Formative: BST construction and search trace table (25 marks)	Individual; markscheme review

Topic 5.2: Tree Traversal and Graph Algorithms 树遍历与图算法

Week 4 — Tree Traversal 树遍历 Shù Biànlì

Lesson	Content	Activities
1	Preorder traversal: root, left, right (前序遍历)	Traverse given trees; write traversal sequence
2	Inorder traversal: left, root, right; BST inorder yields sorted sequence (中序遍历)	Verify: inorder of BST is sorted; prove to peer
3	Postorder traversal: left, right, root; deletion order (后序遍历)	Determine postorder; relate to expression trees
4	Expression trees: constructing from infix; evaluating with postorder	Build tree from expression; evaluate
5	TOK: Tree traversals produce different "readings" of the same structure. Is any traversal more "true" than another? What determines the right way to traverse?	Discussion; connect to hermeneutics

TOK Knowledge Question: Inorder traversal of a BST produces a sorted list. Is this sorting "discovered" (the order was already there) or "created" (the traversal imposes an order)? What does this suggest about the relationship between representation and knowledge?

Week 5 — Graphs 图 Tú

Lesson	Content	Activities
1	Graph terminology: vertex, edge, directed/undirected, weighted, adjacent, degree, path, cycle (顶点, 边, 有向图, 无向图)	Model real networks as graphs; identify properties
2	Graph representations: adjacency matrix, adjacency list; comparison	Represent same graph both ways; compare space/time
3	Graph applications: social networks, navigation, dependency graphs, state machines	Research: how does Google Maps represent roads?
4	Depth-first search (DFS): algorithm, stack-based, recursive; applications	Trace DFS on graph; implement recursively
5	Breadth-first search (BFS): algorithm, queue-based; shortest path in unweighted graphs	Trace BFS; compare to DFS for finding paths

Week 6 — Graph Algorithms 图算法 Tú Suànfǎ

Lesson	Content	Activities
1	Dijkstra's shortest path: algorithm, greedy approach, trace table	Trace Dijkstra on weighted graph; verify shortest path
2	Minimum spanning tree: Prim's and Kruskal's algorithms (conceptual comparison)	Apply both to same graph; compare results
3	Topological sort: concept, DAGs, scheduling applications	Schedule modules with prerequisites; detect cycles
4	Algorithm comparison and selection: which algorithm for which problem?	Scenario cards: choose and justify algorithm
5	Formative: Graph algorithm trace and comparison (30 marks)	Timed; peer review

Topic 5.3: Dynamic Data Structures 动态数据结构 Dòngtài Shùjù Jiégòu

Week 7 — Dynamic Memory and Pointers 动态内存与指针 Dòngtài Nèicún yǔ Zhǐzhēn

Lesson	Content	Activities
1	Dynamic memory allocation: heap vs stack; new/delete (conceptual); memory leaks	Analyse code for memory leaks; discuss garbage collection
2	Pointers/references: indirection, null pointers, dangling pointers	Trace pointer operations; identify dangerous code
3	Self-referential structures: nodes that contain pointers to same type	Diagram self-referential structures; relate to linked lists

Lesson	Content	Activities
4	Dynamic resizing: array lists, hash tables (conceptual); amortised analysis	Calculate cost of dynamic array resizing
5	Consolidation: Dynamic vs static data structures comparison essay	8-mark timed response

Week 8 — Topic 5 Consolidation

Lesson	Content	Activities
1	Revision: Linked lists, stacks, queues — implementation patterns	Code review; common errors analysis
2	Revision: Trees — construction, traversal, BST operations	Speed tracing; peer quizzing
3	Revision: Graphs — representations, DFS, BFS, Dijkstra	Problem-solving carousel
4	Exam technique: 12-mark algorithm design questions	Plan responses; share structures
5	Formative: Topic 5 progress test (40 marks, Paper 1 style)	Timed; self-assessment

Week 9 — Topic 5 End-of-Topic Assessment

Lesson	Content	Activities
1	Revision clinic	Student-led Q&A
2	Assessment: Topic 5 End-of-Topic Test (1 hour, 50 marks)	Formal conditions
3	Test feedback and DIRT	Markscheme analysis
4	Introduction to Topic 6: Operating systems deep dive	Review OS basics; preview HL depth
5	IA check: HL students confirm development progress	One-to-one tutorials

Key Vocabulary (Topic 5)

English	中文	Pinyin	Definition
Linked list	链表	liànbǎo	A dynamic data structure where elements are linked by pointers
Node	节点	jiédiǎn	An element in a linked list or tree containing data and references
Pointer	指针	zhǐzhēn	A reference to a memory location containing another data item
Binary tree	二叉树	èrchà shù	A tree where each node has at most two children

English	中文	Pinyin	Definition
Binary search tree	二叉搜索树	èrchà sōusuǒ shù	A binary tree where left < parent < right
Traversal	遍历	biànlì	Visiting each node in a tree in a specified order
Graph	图	tú	A set of vertices connected by edges
Adjacency matrix	邻接矩阵	línjiē jǔzhèn	A 2D array representing which vertices are connected
Depth-first search	深度优先搜索	shēndù yōuxiān sōusuǒ	Explores as far as possible along each branch before backtracking
Breadth-first search	广度优先搜索	guǎngdù yōuxiān sōusuǒ	Explores all neighbours at current depth before moving deeper
Dynamic allocation	动态分配	dòngtài fēnpèi	Memory allocated at runtime rather than compile time

Teacher Checklist — End of Topic 5:

- ☐ All IB 5.1 – 5.3 syllabus points covered
- ☐ Students can implement linked list, stack, queue in pseudocode
- ☐ Students can traverse trees and construct BSTs
- ☐ Students can represent graphs and apply DFS/BFS/Dijkstra
- ☐ TOK connections documented
- ☐ IA progress monitored for all HL students
- ☐ Victoria Park Academy Shenzhen | IB DP Computer Science | Topic 5 Scheme of Work